THESIS

Javascript Vulnerability Detection using TabM Architectures



RAMA ANDHIKA PRATAMA

23/517333/PA/22175

BACHELOR'S DEGREE PROGRAM
DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF MATHEMATICS AND NATURAL SCIENCES UNIVERSITAS GADJAH MADA
YOGYAKARTA

Year 2026

THESIS

Javascript Vulnerability Detection using TabM Architectures

Diajukan untuk memenuhi salah satu syarat memperoleh derajat
Sarjana Ilmu Komputer



RAMA ANDHIKA PRATAMA
23/517333/PA/22175

BACHELOR'S DEGREE PROGRAM
DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF MATHEMATICS AND NATURAL SCIENCES UNIVERSITAS GADJAH MADA
YOGYAKARTA

Year 2026

HALAMAN PENGESAHAN

SKRIPSI

Javascript Vulnerability Detection using TabM Architectures

Telah dipersiapkan dan disusun oleh

Rama Andhika Pratama

23/517333/PA/22175

Telah dipertahankan di depan Tim Penguji

pada tanggal ...

Susunan Tim Penguji

Nama Pembimbing I	Nama Penguji I
Pembimbing I/Penguji	Penguji
Nama Pembimbing II	Nama Penguji II
Pembimbing II/Penguji	Penguji

Nama Penguji III *(jika ada)*
Penguji

PERNYATAAN

Dengan ini saya menyatakan bahwa skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Yogyakarta, 11 Mei 2026



Rama Andhika Pratama

TABLE OF CONTENTS

HALAMAN PENGESAHAN....2
PERNYATAAN....2
TABLE OF CONTENTS....4
LIST OF TABLES....6
LIST OF FIGURES....7
INTISARI....8
ABSTRACT....10
Chapter 1 INTRODUCTION....1
1.1 Background....1
1.2 Problem Formulation....4
1.3 Research Limitations....4
1.4 Research Objectives....5
1.5 Research Benefits....5
Chapter 2 7
LITERATURE REVIEW....7
2.1 Tabular Software Metrics and Vulnerability Detection....7
2.2 Comparative Analysis of Previous Vulnerability Detection Models....8
2.3 Identified Research Gap and Significance of the Study....13
Chapter 3 14
RESEARCH METHODOLOGY....14
3.1 General Description....14
3.2 Data Acquisition....15
3.2.1 Ferenc et al. (2019) Dataset....15
3.2.2 Viszkok et al. (2021) Dataset....16
3.3 Data Preprocessing....17
3.4 Data Splitting....18
3.5 Class Imbalance Analysis and Handling....20
3.6 Model Implementation....21
3.6.1 TabM (Native Tabular Deep Learning)....21
3.6.2 FT-Transformer....23
3.6.3 Gradient Boosted Decision Trees (GDBT)....26
3.7 Hyperparameter Tuning....28
3.8 Model Evaluation....31
3.8.1 Performance Metrics....31
3.8.2 Efficiency Metrics....33
3.9 Hardware and Software Environment....33

Chapter 4 34
RESEARCH SCHEDULE.....34
REFERENCES.....36
ATTACHMENTS.....38

LIST OF TABLES

Table 2.1 Comparison of Recent Vulnerability Prediction Methodologies……9
Table 3.1 Data Split Configuration for Ferenc's Dataset……...……19
Table 3.2 Data Split Configuration for Viszkok's Dataset……...….19
Table 3.3 Hyperparameter Tuning Configuration for All Three Architectures….29
Table 4.1 Proposed Research Schedule…….34

LIST OF FIGURES

Figure 3.1 Research Pipeline Flowchart……14
Figure 3.2 Class Weight Application on Model Training Pipeline……...21
Figure 3.3 The End-to-End Forward Pass of TabM……...……...23
Figure 3.4 The End-to-End Pipeline of FT-Transformer……...25
Figure 3.5 The End-to-End Pipeline of XGBoost……..……...28

INTISARI

Javascript Vulnerability Detection using TabM Architectures

Oleh

Rama Andhika Pratama 23/517333/PA/22175

JavaScript merupakan salah satu bahasa pemrograman yang paling banyak digunakan dalam sistem perangkat lunak modern, sehingga menjadikannya sebagai target serangan siber yang umum terjadi. Deteksi kerentanan secara otomatis telah menjadi kebutuhan mendasar karena tinjauan kode secara manual tidak mampu mengimbangi kecepatan alur kerja Continuous Integration dan Continuous Deployment (CI/CD). Pendekatan yang ada untuk mendeteksi kerentanan pada fungsi JavaScript menggunakan dataset metrik perangkat lunak berbentuk tabular masih terbatas pada metode machine learning dangkal dan deep learning konvensional, sehingga potensi arsitektur deep learning tabular modern yang dirancang khusus belum pernah dieksplorasi. Penelitian ini mengimplementasikan dan mengevaluasi tiga arsitektur pada dua dataset metrik perangkat lunak tabular JavaScript yang disediakan oleh Ferenc et al. (2019) dan Viszkok et al. (2021), yaitu Gradient Boosted Decision Tree (XGBoost) sebagai baseline machine learning, FT-Transformer sebagai baseline deep learning berbasis attention, dan TabM sebagai model deep learning berbasis ensembel yang efisien secara parameter. Seluruh model dilatih menggunakan pipeline yang konsisten dan dapat direproduksi, yang mencakup praproses data terstandarisasi, pembagian data secara stratifikasi, penyesuaian bobot kelas untuk menangani ketidakseimbangan kelas yang signifikan, serta penyetelan hyperparameter secara sistematis menggunakan Optuna dengan PR-AUC sebagai target optimasi. Performa model dievaluasi menggunakan PR-AUC, F2-Score, dan Matthews Correlation Coefficient, sedangkan efisiensi komputasi diukur melalui latensi inferensi, waktu pelatihan ulang, ukuran model, dan penggunaan memori puncak. Evaluasi komparatif ini bertujuan untuk menentukan apakah TabM mampu mencapai performa prediktif yang kompetitif dibandingkan FT-Transformer dengan biaya komputasi yang jauh lebih rendah, serta apakah salah satu arsitektur deep learning dapat melampaui

baseline XGBoost pada tugas deteksi kerentanan berbasis data tabular ini.

ABSTRACT

Javascript Vulnerability Detection using TabM Architectures

by

Rama Andhika Pratama 23/517333/PA/22175

JavaScript is one of the most widely used programming languages in modern software systems, making it a frequent target of cyber-attacks. Automated vulnerability detection has become essential as manual code reviews cannot keep pace with the speed of Continuous Integration and Continuous Deployment (CI/CD) pipelines. Existing approaches for detecting vulnerabilities in JavaScript functions using tabular software metrics remain limited to traditional shallow machine learning and conventional deep learning methods, leaving the potential of recent native tabular deep learning architectures unexplored. This study implements and evaluates three architectures on two JavaScript tabular software metrics datasets provided by Ferenc et al. (2019) and Viszkok et al. (2021): a Gradient Boosted Decision Tree (XGBoost) as the machine learning baseline, FT-Transformer as an attention-based deep learning baseline, and TabM as a parameter-efficient ensemble-based deep learning model. All models are trained under a consistent and reproducible pipeline that includes standardized preprocessing, stratified data splitting, class weight adjustment to handle severe class imbalance, and systematic hyperparameter tuning using Optuna with PR-AUC as the optimization target. Model performance is evaluated using PR-AUC, F2-Score, and Matthews Correlation Coefficient, while computational efficiency is measured through inference latency, retraining time, model size, and peak memory usage. The comparative evaluation aims to determine whether TabM can achieve competitive predictive performance relative to FT-Transformer while offering substantially lower computational cost, and whether either deep learning

architecture can surpass the XGBoost baseline on this tabular vulnerability detection task.

Chapter 1
INTRODUCTION

1.1 Background

Modern systems are commonly developed using JavaScript, which makes the language one of the main targets for cyber-attacks. If security flaws exist in the system, data breaches will likely occur, which can lead to massive economic damage. As an illustration, software vulnerabilities cost the US economy an estimated $60 billion annually in productivity loss and patch redeployment, where each individual cyber-attacks causing damages reaching up to $8.75 billion (Anwar et al., 2020). The modern shift toward Continuous Integration and Continuous Deployment (CI/CD) approach increases the risks further. In order to maintain a high-speed environment in CI/CD, highly sensitive credentials are stored in a single, centralized location. If it is successfully compromised by attackers, they will have access to the whole deployment infrastructure (Pan et al., 2024). As development gets more efficient, vulnerabilities are more likely to occur and traditional security systems are unable to keep up.

Since manual code reviews are unable to keep up with the high-speed environment, numerous efforts have been made to create automated vulnerabilities detection. Traditional static analysis tools were originally built to address software security risk issues. However, these tools often produce an overwhelming volume of raw warnings with high false-positive rates. Therefore, developers had to manually review the code again which defeats the purpose of having an efficient automated code detector (Cheirdari & Karabatis, 2018). This makes machine learning a promising and more robust alternative by learning vulnerability patterns from historical data. Alternative methods such as machine learning and deep learning are excellent candidates for vulnerability detection because they learn vulnerability patterns from historical data. Machine learning offers a lightweight, efficient, and accurate approach, while deep learning provides rich data representations.

However, these methods can not be applied directly to the source code. In

order to apply these methods in software vulnerability detection, the source code must be translated first into a structured format. Researchers addressed this issue by extracting the characteristics of the source code – known as Software Metrics – from various source entities such as files, functions, or slices. As an illustration, these software metrics include numerical measurements of code structure (such as cyclomatic complexity, size, coupling, and Halstead metrics) and in some cases historical or project-level characteristics. These metrics are then fed into vulnerability prediction models as features for the model to learn (Medeiros et al., 2020; Zagane et al., 2020; Pakshad et al., 2023). This extraction process transforms the complex source code repositories into lightweight, tabular datasets.

An example of this approach is by Ferenc et al. (2019) where he constructed a tabular JavaScript Vulnerability Dataset where he extracted the software metrics of diverse corpus from vast amounts of open-source JavaScript repositories into static source code metrics. The dataset was later expanded by Viszkok et al. (2021) where they added historical process metrics to the original dataset. Both studies evaluated their dataset using several traditional shallow machine learning and deep learning methods as their baseline. Therefore, their exploration of advanced machine learning and deep learning approaches were still limited, leaving a significant research gap regarding how advanced machine learning methods and neural networks process their specific tabular data of software metrics.

Recent advances in native tabular deep learning have produced architectures designed to address various computational and performance limitations within the field. One such architecture, FT-Transformer (Feature Tokenizer + Transformer), was proposed by Gorishniy et al. (2021) as a general-purpose baseline for standard supervised classification and regression tasks on tabular data. It adapts the Transformer architecture by transforming all features into embeddings and applying a stack of Transformer layers, allowing every layer to operate at the feature level of one object. However, despite its strong predictive performance, FT-Transformer has been noted to be

computationally expensive, as the self-attention mechanism scales poorly with the number of features and training samples, a limitation that constrains its practicality in resource-sensitive or time-critical environments.

To address this, Gorishniy et al. (2025) introduced TabM, a parameter-efficient tabular deep learning architecture that uses an ensembling mechanism allowing a single neural network to simulate an ensemble of MLPs. Rather than relying on attention across features, TabM produces multiple predictions simultaneously while sharing the majority of the model's underlying parameters, achieving competitive accuracy at a fraction of the computational cost. Given these properties, TabM presents a compelling candidate for application in software security contexts, specifically JavaScript vulnerability prediction, where both accuracy and efficiency are critical. Its lightweight nature may make it particularly well-suited for integration within rapid, high-throughput CI/CD pipelines, where inference speed and low overhead are as important as predictive reliability.

Therefore, this research proposes the implementation and the evaluation of FT-Transformer and TabM on the JavaScript vulnerability prediction task using the tabular software metrics datasets. By benchmarking these two architecturally distinct deep learning approaches against the shallow machine learning baselines established in prior work, this study directly addresses the research gap left by their limited exploration of advanced deep learning approaches on the specific datasets. Given the escalating security risks introduced by modern CI/CD pipelines and the inefficiency of manual code reviews and traditional static analysis tools, a more robust and accurate vulnerability prediction model is critically needed. The complementary nature of FT-Transformer's attention-based feature interactions and TabM's parameter-efficient ensemble mechanism provides a compelling basis for determining which architecture captures the vulnerability patterns within software metrics better and faster. Ultimately, the findings of this research aim to contribute actionable insights toward building more reliable automated vulnerability detection systems for JavaScript, supporting the broader

goal of reducing the economic and operational damage caused by software security flaws.

1.2 Problem Formulation

The current approaches for JavaScript vulnerability detection, particularly on tabular software metrics dataset, are still limited to traditional shallow machine learning and deep learning methods. This study addresses that gap by applying TabM, a recent native parameter-efficient tabular deep learning architecture, to investigate whether it can match or surpass the performance of existing methods while remaining practical for high-speed CI/CD environments. To contextualize its performance, TabM will be benchmarked against FT-Transformer as a deep learning baseline, and against a Gradient Boosted Decision Tree method, specifically XGBoost, which represents the current performance ceiling for machine learning approaches on tabular data. This comparison aims to determine whether TabM can deliver a favorable balance between predictive accuracy and computational efficiency, making it a practical option for real-world vulnerability detection pipelines.

1.3 Research Limitations

To maintain a focused and achievable research objective, the scope and limitations of this study are defined as follows:

- The scope is exclusively limited to the JavaScript programming language and will not be extended to other languages such as C/C++ or Python.
- This study is strictly limited to the tabular datasets constructed by Ferenc et al. (2019) and Viszkok et al. (2021). This constraint is necessary because of the current scarcity of metric-based JavaScript vulnerability datasets. While various vulnerability datasets exist, most of them provide raw source code instead of structured numerical features. As of June 2026, the Ferenc and Viszkok datasets are the only publicly available benchmarks that fit the requirements of this research.

- The comparison is confined to the numerical features within the tabular datasets, specifically the software metrics. Although the source code of every data exists within the datasets, it will not be utilized nor applied to the training process.
- The evaluation is strictly bounded to three specific architectures: a Gradient Boosted Decision Tree (GBDT) baseline, attention-based tabular deep learning (FT-Transformer), and parameter-efficient ensemble-based deep learning (TabM). The study will not explore or implement other architectures.

1.4 Research Objectives

The specific objectives of this research are:

1. To implement and evaluate the three different architectures—a Gradient Boosted Decision Tree, an attention-based deep learning architecture (FT-Transformer), and a parameter-efficient ensemble architecture (TabM)—on the JavaScript tabular software metrics datasets provided by Ferenc et al. (2019) and Viszkok et al. (2021).
2. To benchmark the predictive performance of the three architectures to determine which model most accurately and effectively captures vulnerability patterns within the datasets.
3. To see whether the computational trade-off for the performance gains of the richer representation in FT-Transformer and TabM is justified compared to the GDBT baseline.

1.5 Research Benefits

The benefits derived from this research are divided into two categories:

- **Theoretical Benefits:** This study contributes empirical evidence to the underexplored intersection of native tabular deep learning and software vulnerability detection. By benchmarking the different architectures, this research provides a reproducible academic framework. It clarifies the theoretical viability and architectural trade-offs of using deep learning to capture complex vulnerability patterns using software metrics.

- **Practical Benefits**: The findings of this research offer guidance to software engineers, security tool developers, and DevOps teams seeking automated vulnerability detection methods for their pipeline environment. By evaluating the trade-offs between the different architectures, they can determine whether the computational cost of deploying richer, complex representations is justified, or if a lightweight, efficient model provides a more practical balance of speed and accuracy for their high-speed environments.

Chapter 2
LITERATURE REVIEW

2.1 Tabular Software Metrics and Vulnerability Detection

When implementing predictive models for software engineering, such as vulnerability detection, tabular data structured in standard rows and columns remains highly relevant. This format often utilizes metric-based features that were extracted from the source code of various repositories. It bypasses the need for extensive structural feature engineering, making it computationally efficient and lightweight for real-time and rapid CI/CD pipelines. The JavaScript vulnerability datasets introduced by Ferenc et al. (2019) and Viszkok et al. (2021) rely on software metrics and serve as the benchmark that demonstrates the efficacy of the lightweight tabular paradigm in detecting vulnerable functions in this study.

In the world of tabular data processing, Gradient Boosted Decision Tree (GBDT) algorithms such as XGBoost remain as robust baselines. Their strong performance came from their intrinsic ability to handle diverse and heterogeneous feature distributions through successive space partitioning when handling data (Ye et al., 2025). While GBDTs remain highly effective, modern deep learning architectures are increasingly narrowing this performance gap by demonstrating a superior capacity to map higher-order interactions and complex non-linear feature compositions within tabular datasets (Ye et al., 2025).

However, the narrowing performance gap between deep learning and GBDTs does not mean that every deep learning architecture will work well on every dataset and task. It is still necessary to identify an architecture that is appropriate for the specific data type and problem at hand. Sheneamer (2024) provides a relevant example by applying a Convolutional Neural Network (CNN) to tabular software metric data for vulnerability detection. Although the approach achieved good accuracy, it carries a fundamental flaw: CNNs are designed to recognize spatial patterns, making them most naturally suited for pixel-based data such as images. When applied to tabular data, where features have no spatial relationship with one another, this architectural bias becomes a limitation that may

prevent the model from fully capturing the underlying patterns in the data.

This leaves a gap for exploration and the need for deep learning approaches that are genuinely designed for tabular data, while also being efficient enough to support the fast-paced nature of CI/CD pipelines. TabM is proposed as a method that meets both of these requirements. Instead of introducing heavy computational overhead, TabM uses parameter-efficient ensembling, where a single model behaves like an ensemble of independent networks that share most of their parameters, producing multiple predictions at once without significant memory costs (Gorishniy et al., 2025). This makes TabM a well-suited choice for the lightweight and real-time demands of the vulnerability detection task in this study.

2.2 Comparative Analysis of Previous Vulnerability Detection Models

The foundation for lightweight JavaScript vulnerability prediction using software metrics was established by Ferenc et al. (2019) and subsequently expanded by Viszkok et al. (2021). By combining static source code features with historical process metrics into a standardized software metrics tabular dataset, they provided a highly lightweight and efficient alternative to complex code analysis, significantly advancing the field of automated vulnerability detection. However, their methodologies are limited and relied exclusively on traditional, shallow machine learning algorithms (such as Random Forest) and deep learning algorithms (such as Deep Neural Network). This leaves a gap for future exploration where advanced models could further improve predictive performance on such tabular software metrics datasets.

Several studies have attempted to build upon these datasets. Sheneamer (2024) expanded the exploration by applying a Convolutional Neural Network (CNN). However, this approach is fundamentally mismatched, as CNNs are spatially biased models better suited for image data, and the study also suffered from data leakage due to improper dataset splitting. Kalouptsoglou et al. (2022) extended the exploration further by combining the software metrics with text features mined directly from source code into a single unified model. However,

their results did not achieve higher results. Mosolygó et al. (2021) took a different direction by constructing Word2Vec vector similarity with custom filtering rules, offering built-in explainability and the ability to provide evidence for each flagged instance. However, their approach did not incorporate the software metrics component of the dataset and does not align with our research.

Other studies have explored advanced deep learning approaches. However, most of them suffer from being computationally expensive, which makes them unsuitable for high-speed CI/CD environments. Shwartz-Ziv and Armon (2022) demonstrated that combining deep learning models with XGBoost outperformed individual deep models, standalone XGBoost, and classical ensembles alike, but it requires a large amount of computational resources. Similarly, Gorishniy et al. (2021) proposed the Feature Tokenizer Transformer (FT-Transformer), which produces richer and deeper feature representations while remaining relatively interpretable. However, it still remains computationally expensive for lightweight deployment. The proposed approach in this study therefore aims to identify models that balance predictive accuracy with computational efficiency, making them practical for JavaScript vulnerability detection within rapid CI/CD pipelines. A comparative analysis of the algorithms, datasets, methods, and their respective advantages and disadvantages is summarized in Table 2.1.

Table 2.1 Comparison of Recent Vulnerability Prediction Methodologies

Author & Year	**Algorithm / Architecture**	**Dataset Size & Type**	**Imbalance Handling**	**Advantages and Disadvantages**
Ferenc et al. (2019)	8 ML algorithms, including KNN, Random Forest and 2 DL algorithms	12,125 JavaScript functions using only static source code metrics	Tested random under-sampling and over-sampling at various ratios	(+) Introduced the foundational tabular dataset mapping JS source code to static software metrics

				(-) Relied on traditional shallow machine learning and deep learning applications
Viszkok et al. (2021)	8 ML algorithms, including KNN, Random Forest and 2 DL algorithms	8,038 JavaScript functions using static and process metrics	Tested random under-sampling and over-sampling at various ratios	(+) Expanded the tabular dataset by adding historical process metrics. (-) Still relied on traditional shallow machine learning and deep learning applications
Sheneamer (2024)	Stacked 1D CNNs including VGG16, VGG19, AlexNet, and ResNet	30,367 JavaScript functions across 3 tabular metric datasets (Ferenc et al., Viszkok et al., and Apache Tomcat)	Applied random under-sampling and random over-sampling techniques	(+) Expanded the exploration of the dataset by applying an advanced neural network method. (-) Relies on spatially biased CNNs (-) Introduces pre-split data leakage.

Shwartz-Ziv and Armon (2022)	Compared 4 tabular deep learning models (TabNet, DNF-Net, NODE, 1D-CNN) against XGBoost	11 diverse tabular datasets containing up to 1,000,000 samples	Not explicitly addressed	(+) The ensemble combining both deep learning models and XGBoost outperformed individual deep models, standalone XGBoost, and ensembles made strictly of classical models (-) Demonstrated the ensemble of XGBoost and deep models are computationally expensive
Gorishniy et al. (2021)	Proposed FT-Transformer and ResNet architectures for tabular data	11 diverse public tabular datasets up to 1,200,192 samples	Not explicitly addressed in the methodology	(+) Richer and deeper representation making interpretation easier and efficient (-) FT-Transformer demands high computational resources

Kaloupts oglou et al. (2022)	ML, DL (CNN), and ensemble models	Processed Ferenc et al. (2019) Dataset: 12,106 open-source JavaScript functions	Random over-sampling and under-sampling	(+) Extended the exploration further by combining the original software metrics and the text features mined from the source codes into a unified model (-) The combined models did not achieved higher result
Mosolygó et al. (2021)	Word2vec vector similarity with custom filtering rules	Ferenc et al. (2019) Dataset processed into 186 real-world vulnerability fixes from 91 JavaScript projects	Inherently unaffected; relies strictly on similarity to reference lines	(+) Has built-in explainability and able to provide proof for each flagged data (-) The model did not rely on the software metrics data

2.3 Identified Research Gap and Significance of the Study

As synthesized in Table 2.1, a critical gap exists in the current application of advanced machine learning and deep learning methods for JavaScript vulnerability detection using tabular software metrics datasets. Several studies have attempted to address this by building more advanced models on the dataset. However, these efforts still remain limited. Several approaches are fundamentally mismatched for tabular data due to their reliance on spatially biased architectures, others are misaligned with the research scope as they disregard the software metrics feature entirely, and some simply fail to achieve an improvement over existing baselines. On the other hand, studies that have applied advanced deep learning to tabular datasets more broadly still share a common limitation: expensive computational cost, which makes them impractical for deployment in high-speed CI/CD pipelines where lightweight and efficient is essential.

To address this gap, this study proposes TabM, a recent native tabular method designed for processing tabular data efficiently, and evaluates it on both tabular datasets. Introduced by Gorishniy et al. (2025), TabM represents a significant advancement in native tabular deep learning through its use of parameter-efficient ensembling, which allows a single model to function as multiple neural networks simultaneously without demanding excessive memory or computation. This property makes TabM well-suited for integration into rapid CI/CD pipelines. To benchmark its performance, TabM is compared against two baselines. The FT-Transformer serves as the deep learning baseline, offering richer and deeper feature representations that support more efficient interpretation, though it remains computationally expensive. A Gradient Boosted Decision Tree (GBDT) model is also included as the machine learning baseline, representing the established performance ceiling for tabular data processing, in order to assess whether modern deep learning approach can compete with, or surpass, the dominant approach for structured tabular data.

Chapter 3
RESEARCH METHODOLOGY

3.1 General Description

This chapter describes the complete structure of the research methodology employed in this study. The research follows a structured pipeline designed to produce a fair and reproducible comparative evaluation. At a high level, the workflow proceeds through nine sequential stages: (1) data acquisition, (2) data preprocessing, (3) stratified data split, (4) class imbalance handling on training set, (5) model implementation, (6) hyperparameter tuning, (7) model training, (8) model evaluation, and (9) comparative analysis.

Figure 3.1 presents the research flowchart that illustrates the research's end-to-end pipeline. The process begins with data acquisition, explaining about the characteristics of each dataset. The datasets are then preprocessed and partitioned into training, validation, and test sets. Before the models are configured, the imbalance inside the datasets will be adjusted using class weight. The models are then configured, tuned, and trained using the specified evaluation protocol. Finally, the models are evaluated on the held-out test set using a common suite of classification metrics, and their results are compared.

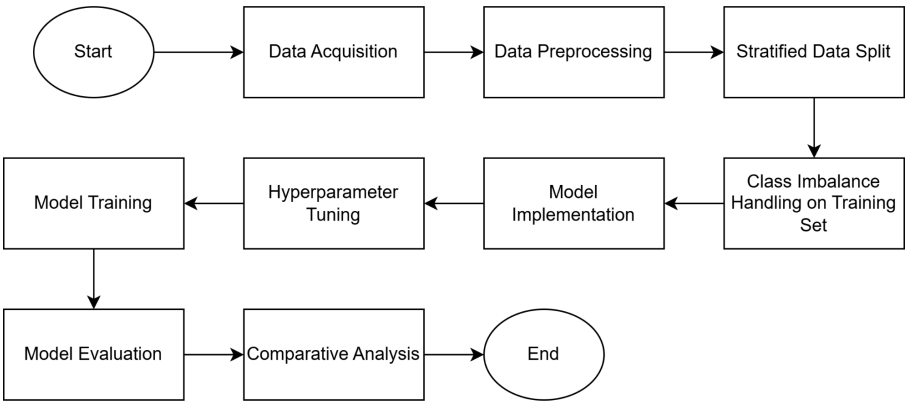


Figure 3.1 Research Pipeline Flowchart

3.2 Data Acquisition

3.2.1 Ferenc et al. (2019) Dataset

The first dataset was constructed by Ferenc et al. (2019) as part of a study investigating whether static source code metrics could serve as effective predictors of security vulnerabilities in JavaScript functions. To build the dataset, the authors mined two publicly available vulnerability databases — the Node Security Platform (nsp) and the Snyk Vulnerability Database — and cross-referenced their entries with GitHub repositories to locate the specific commits that fixed each reported vulnerability. For each vulnerability, the corresponding patch file was extracted and mapped to the JavaScript functions it modified. Functions in the pre-fix version of the code (i.e., before the security patch was applied) that fell within the range of the patch's changed lines were marked as vulnerable, while all remaining functions in those same projects were marked as non-vulnerable. Test functions, identified by their location under "test" folders, were excluded to prevent them from distorting the prediction models. This process resulted in a dataset of 12,125 JavaScript functions drawn from over 300 open-source Node.js projects, of which 1,496 (approximately 12.3%) are labeled as vulnerable and 10,629 as non-vulnerable.

For each function, 35 static source code metrics were computed using two complementary static analysis tools: OpenStaticAnalyzer and escomplex. Static source code metrics are structural properties derived by analyzing the source code at a single point in time without executing it. The metrics captured span several dimensions of code quality and complexity, including size metrics such as Lines of Code (LOC) and Logical Lines of Code (LLOC), complexity metrics such as Cyclomatic Complexity (McCC/CYCL) which counts the number of independent execution paths through a function, nesting metrics such as Nesting Level (NL), structural metrics such as the Number of Parameters (NUMPAR/PARAMS), clone-related metrics such as Clone Coverage (CC) and Clone Instances (CI), comment and documentation metrics such as Comment Density (CD) and Comment Lines of Code (CLOC), and the full suite of Halstead software science metrics — including Halstead Volume (HVOL), Halstead Difficulty (HDIFF),

Halstead Effort (HEFF), and Halstead Bugs (HBUGS) — which characterize the information content and estimated error proneness of the code based on its operator and operand counts.

The target label is a binary indicator stored in the final column of the dataset. A value of 1 denotes that the function was vulnerable, meaning it was directly modified by a known security-fixing commit prior to the fix being applied. A value of 0 denotes that the function was non-vulnerable, meaning it was not implicated in any vulnerability fix within the same codebase. This binary labeling scheme makes the dataset directly suitable for supervised binary classification.

3.2.2 Viszkok et al. (2021) Dataset

To avoid biases that may arise from relying on a single dataset, this study also employs a second dataset provided by Viszkok et al. (2021). This dataset was originally constructed by Ferenc et al. (2019) and was expanded by Viszkok et al. to include historical process data derived from each project's version control history.

The dataset contains function-level records extracted from the same corpus of over 300 open-source JavaScript repositories used in the original study. However, because the extraction of process metrics requires a complete and accessible Git history for each project, not all functions from the original 12,125-function dataset could be retained — projects whose repositories were no longer accessible or whose version control history was incomplete had to be excluded. As a result, the extended dataset contains 8,038 functions, a reduction of roughly 33% from the original. This creates a trade-off that adding historical data gives the model more depth and features to work with, but it shrinks the overall size of the dataset.

For each function, the extended dataset preserves the original 42 static source code metrics. This count is slightly higher than the 35 metrics in the Ferenc et al. dataset because Viszkok et al. incorporated additional static metrics during the extension process, bringing the static metric total from 35 to 42. In addition,

the dataset introduces 19 process metrics, also referred to as historical change or process metrics. It captures the development history of each function as recorded in its Git commit log. These include the Number of Modifications (NOMOD), which counts how many times a function was changed over its lifetime; the Number of Contributors (NOContr), which captures how many distinct developers modified it; Code Churn (CChurn), defined as the cumulative sum of lines added minus lines deleted; the Average Time Between Changes (AVGTBC); and various aggregated counts of added, deleted, and modified lines across commits. Together, the 42 static and 19 process metrics create a combined feature space of 61 predictors per function, making this a substantially richer representation than the original dataset.

This extended dataset serves as the second baseline for the experiments in this study. The richer feature set is expected to improve model performance, which was confirmed in the original study where Viszkok et al., reported significant gains in F-measure, precision, and recall when process metrics were added. Although the reduction in training samples may introduce its own flaws. The binary target label remains identical to that of the Ferenc et al. dataset: a value of 1 indicates that the function was vulnerable prior to a known security fix, while a value of 0 indicates that it was non-vulnerable. The class distribution also remains heavily imbalanced, with vulnerable functions constituting a minority of the total records.

3.3 Data Preprocessing

To ensure a fair and rigorous comparison, a standardized preprocessing pipeline is applied uniformly across all three architectures. Because the features derived from both datasets are strictly continuous numerical metrics, categorical encoding is not required. The preprocessing pipeline consists of the following three sequential steps:

- **Feature Selection / Filtering:** Identifier columns (*name, longname, path, full_repo_path, hash*), zero-variance features (*WarningBlocker, WarningCritical, WarningInfo, WarningMinor, CLLC*), and one feature from each perfectly collinear pair (*HTIME, HVOL, end_column, end_line,*

start_column, start_line, CLLC/CLC) are removed from the dataset. These features carry no predictive signal and their removal are necessary for standard data hygiene practice rather than a model-specific decision, ensuring that all three architectures benefit equally from a cleaner feature space.

- **Positional Columns Isolation:** Positional features such as *line, column, endline,* and *endcolumn* are excluded from model training. Although these columns are moderately correlated to the target variable (*Vuln* ≈ 0.20), this relationship is likely an artifact of the dataset's construction rather than a generalizable signal. Retaining them would inflate the performance scores across all models and give massive advantage to XGBoost, which relies heavily on any available correlation, introducing an unfair advantage.
- **Symmetrical Feature Normalization:** A QuantileTransformer is applied to all remaining numeric features uniformly across all three models. This step is the main fix used to ensure fairness across the models.. While tree-based models such as XGBoost are invariant to monotonic feature transformations, gradient-based deep learning models such as TabM and FT-Transformer are sensitive to the distributional properties of their inputs. Applying this normalization ensures that the deep learning models are not penalized by skewed raw input distributions, while also leaving XGBoost's behavior unaffected.

3.4 Data Splitting

The dataset is partitioned into three non-overlapping subsets: a training set, a validation set u, and a test set used. The split is performed using stratified sampling to preserve the original class ratio in all three subsets. Table 3.1 presents the data split configuration for Ferenc’s dataset and Table 3.2 presents the data split configuration for Viszkok’s Dataset.

Table 3.1 Data Split Configuration for Ferenc's Dataset

Dataset Split	Total Samples	Vulnerable	Non-Vulnerable
Training (60%)	~7,275	~898	~6,377
Validation (20%)	~2,425	~299	~2,126
Test (20%)	~2,425	~299	~2,126
Total	12,125	1,496 (12.34%)	10,629 (87.66%)

Table 3.2 Data Split Configuration for Viszkok's Dataset

Dataset Split	Total Samples	Vulnerable	Non-Vulnerable
Training (60%)	~4,823	~576	~4,247
Validation (20%)	~1,608	~192	~1,416
Test (20%)	~1,608	~192	~1,416
Total	8,038	960 (11.94%)	7,078 (88.06%)

The 60/20/20 stratified split is selected because of the data imbalance across both datasets. Since the minority class only populates 12% of the data, it creates a small number for validation and test samples, which makes it difficult for reliable evaluation. An 80/10/10 split would yield only 96 vulnerable samples in each evaluation set for Viszkok's Dataset, producing a 95% confidence interval width of ±0.103, which is too wide to support reliable comparisons between models. A 60/20/20 split instead provides 192 and 299 vulnerable samples in the validation and test sets for the Viszkok and Ferenc datasets respectively, narrowing the confidence interval to ±0.071 and ±0.057, a 31% reduction in evaluation uncertainty. Allocating 60% for training also ensures that TabM and FT-Transformer receive a minimum of approximately 4,800 training samples, which is near the lower bound required for gradient-based tabular models to learn stable representations. Applying a consistent split ratio across both datasets ensures that any observed performance differences reflect the intrinsic characteristics of the data rather than differences in training volume. All splits are performed with a fixed random seed (seed = 42) to ensure reproducibility.

3.5 Class Imbalance Analysis and Handling

The dataset suffers from a severe class imbalance, with approximately 12% of samples representing vulnerable functions and the remaining 88% representing non-vulnerable functions. This inequality poses a significant challenge when doing model training. A naive classifier could predict every sample as non-vulnerable and it would still achieve an 88% accuracy while completely failing at the intended detection task.

To address this issue, this study employs class weight adjustment, a technique grounded in cost-sensitive learning (Elkan, 2001), to mitigate the class imbalance problem. Cost-sensitive learning modifies the training objective so that the algorithm penalizes the model for misclassifying a minority sample more than a majority sample. Consequently, the loss function for each model is modified to assign a proportionally higher penalty to the vulnerable class. The weight assigned to the minority class is calculated using the equation below:

$$W_{minority} = \frac{N_{total}}{C \times N_{Minority}} \qquad (3.1)$$

where N_{total} is the total number of samples, $N_{minority}$ is the number of vulnerable samples, and C is the number of classes, which in this case would be equal to 2 because our task is binary classification. This method was chosen over SMOTE and oversampling methods because they distort feature space differently for different model architecture making an unfair comparison environment. In addition, for imbalanced tabular software metrics, retaining the original class imbalance and training standard classifiers without synthetic oversampling can be competitive or superior to SMOTE-based rebalancing, which may fail to adequately represent the minority distribution (Sakho et al., 2024).

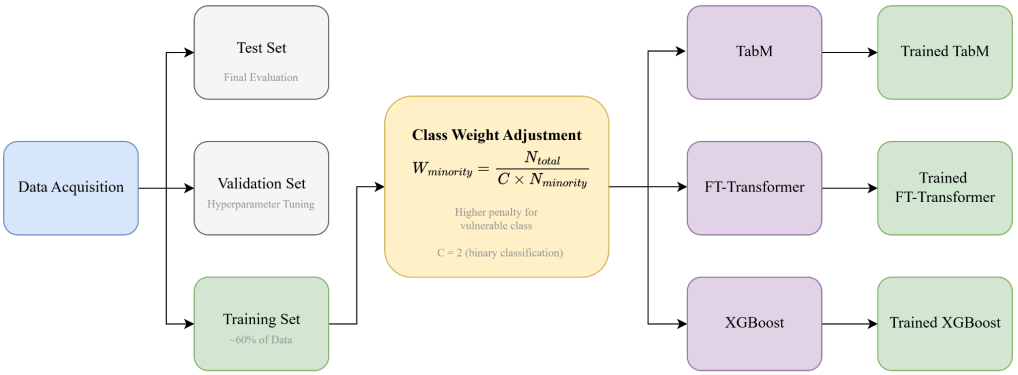


Figure 3.2 Class Weight Application on Model Training Pipeline

3.6 Model Implementation

3.6.1 TabM (Native Tabular Deep Learning)

Each JavaScript function in the Ferenc et al. (2019) and Viszkok et al. (2021) datasets enters the TabM model as a single standardized numerical vector $x \in \mathbb{R}^n$, where n is the number of dimensions corresponds to amount of software metrics being used. corresponds to the number of software metrics available in each respective dataset. At the start of the forward pass, this input vector is immediately replicated k times to produce a parallel input batch $X \in \mathbb{R}^{k \times n}$, where each row is an identical copy of x. This replication step is the foundation of TabM's parameter-efficient ensembling: by constructing k parallel copies of the input from a single sample, TabM allows all k implicit ensemble members to be processed simultaneously in one forward pass, eliminating the need for k separate model executions.

Rather than building k entirely separate networks, all k input streams are routed through a single shared MLP backbone. The backbone's primary weight matrix $W \in \mathbb{R}^{d \times d}$ is shared across all members and handles the main computational work. To introduce diversity between members without duplicating W, each ensemble member $i \in \{1, \ldots, k\}$ is assigned a set of lightweight, non-shared rank-1 adapter vectors s_i, r_i, $b_i \in \mathbb{R}^d$. The linear transformation for member i at any given layer is computed as:

$$\ell_i(x_i) = r_i \odot (W(s_i \odot x_i)) + b_i \quad (3.2)$$

where $\odot$ denotes the Hadamard (elementwise) product, s_i is the input adapter that scales the incoming representation before it is multiplied by W, r_i is the output adapter that scales the result, and b_i is a member-specific bias vector. This formulation is mathematically equivalent to giving each member its own unique weight matrix $W_i = W \odot (r_i s_i^T)$, but without explicitly constructing it — reducing the per-member memory cost from $O(d^2)$ to $O(d)$. Each of the k streams therefore receives a distinctly transformed view of the n input metrics, achieving representational diversity at negligible additional memory cost.

After the linear transformation at each layer, standard non-linear operations are applied to each of the k streams strictly independently. Specifically, a ReLU activation is applied elementwise to each stream's output, followed by Dropout regularization applied separately to each stream's activated representation. This per-stream independence is essential: because the non-linearities and stochastic masking are never shared across members, each of the k simulated ensemble members traces a distinct, non-linear pathway through the n-dimensional software metric space across all N hidden blocks of the backbone. This separation ensures that the ensemble members learn complementary feature combinations from the static and process metrics, which the final aggregation step then leverages to produce more robust predictions than any single member could achieve on its own.

After passing through all N hidden blocks, each of the k parallel streams is fed into a shared linear prediction head, producing k independent scalar logits $\{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_k\}$. These k logits are then aggregated by computing their arithmetic mean, yielding a single unified logit $\bar{y} = (1/k) \Sigma \hat{y}_i$. This averaged logit is passed through a Sigmoid activation function, $\sigma(\bar{y}) = (1 + e^{-\bar{y}})^{-1}$, which maps the output to the interval (0, 1) and produces the final vulnerability probability for the input function. A classification threshold of 0.5 is then applied: if $\sigma(\bar{y}) \geq 0.5$, the function is classified as vulnerable (label = 1); otherwise, it is classified as non-vulnerable (label = 0). The complete end-to-end data flow described across

all four stages — from input expansion through shared backbone processing to final aggregation — is illustrated in Figure 3.3.

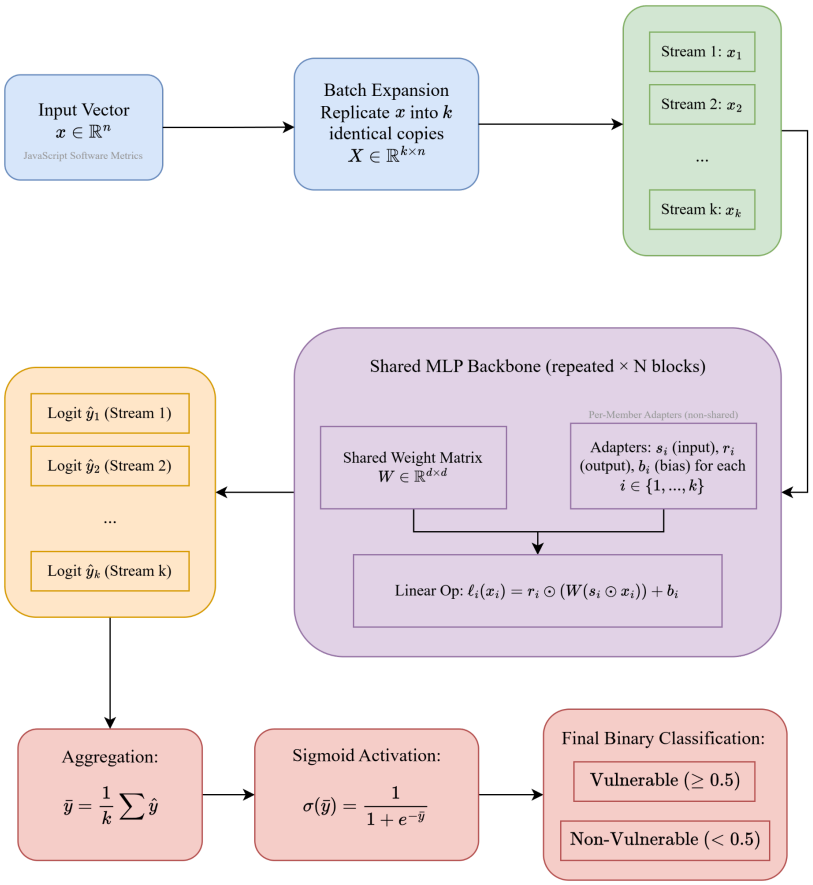


Figure 3.3 The End-to-End Forward Pass of TabM

3.6.2 FT-Transformer

FT-Transformer is included in this study as the deep learning baseline because it produces richer, higher-order feature representations than conventional MLP-based models, which supports more transparent and efficient interpretation of the learned vulnerability patterns. By transforming every feature into an embedding and letting self-attention mechanisms capture pairwise interactions across all software metrics, FT-Transformer can model complex non-linear relationships that shallower architectures may miss. However, this expressive capacity comes with a trade-off of a high computational cost. The architecture demands significantly more memory and training time compared to both XGBoost

and TabM, making it a computationally expensive choice for deployment in a rapid CI/CD pipeline. For this reason, FT-Transformer serves as the interpretability-focused deep learning reference point in this study: the goal is to empirically determine whether the richer representations it produces justify the additional computational overhead, or whether a more parameter-efficient architecture such as TabM can achieve comparable performance at a fraction of the cost.

FT-Transformer, proposed by Gorishniy et al. (2021) in *Revisiting Deep Learning Models for Tabular Data*, adapts the Transformer architecture for tabular data through two sequential components: a Feature Tokenizer and a standard Transformer backbone. In this study, each JavaScript function is represented as a numerical feature vector — either 35 features from the Ferenc et al. (2019) dataset or 61 features from the Viszkok et al. (2021) dataset, depending on which dataset is being processed. The Feature Tokenizer converts each scalar feature into a dense embedding vector by applying a per-feature linear projection combined with a learnable feature bias. This transforms the raw software metric vector into a sequence of token embeddings of shape $k \times d$, where k is the number of features and d is the embedding dimension. This tokenization step allows every feature to be treated as an individual semantic unit, enabling the attention mechanism in subsequent layers to reason about relationships between features rather than aggregating them into a single dense representation at the input level.

Once the feature embeddings are constructed, a special [CLS] (classification) token is prepended to the sequence and the combined sequence of $k + 1$ tokens is passed through L stacked Transformer layers. Each layer applies multi-head self-attention followed by a position-wise feed-forward network, with layer normalization and residual connections following the PreNorm variant for more stable gradient flow. The self-attention mechanism allows every feature token to attend to every other feature token at each layer, enabling the model to capture global interactions across all 35 or 61 software metrics simultaneously. After all Transformer layers have been applied, the final representation of the

[CLS] token serves as the aggregated function-level representation. This representation is passed through a linear prediction head activated by a sigmoid function, producing the vulnerability probability for the input function. A classification threshold of 0.5 is then applied: if $\sigma(\hat{y}) \geq 0.5$, the function is classified as vulnerable; otherwise it is classified as non-vulnerable.

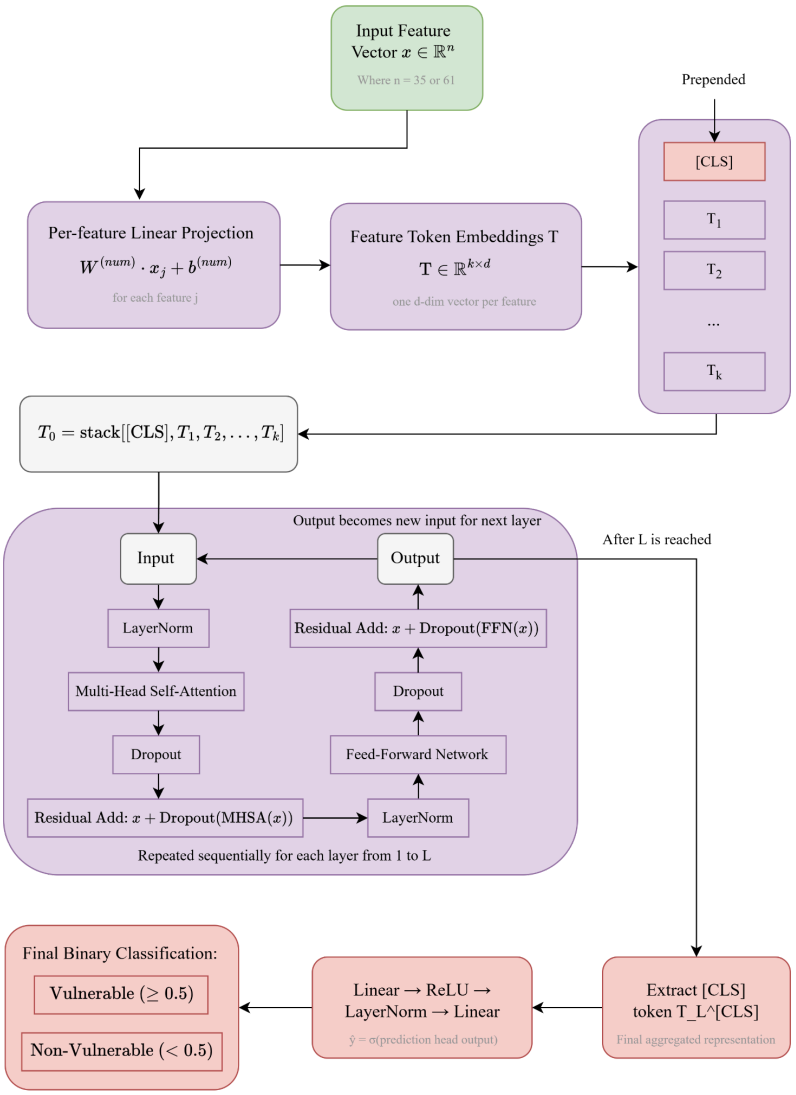


Figure 3.4 The End-to-End Pipeline of FT-Transformer

3.6.3 Gradient Boosted Decision Trees (GDBT)

XGBoost is selected as the representative of Gradient Boosted Decision Tree baseline for this study because GBDT algorithms collectively represent the established performance ceiling for tabular data processing. Their competitive strength derives from their intrinsic ability to handle diverse and heterogeneous feature distributions through successive, greedy space partitioning — each new tree is fit to the negative gradient of the loss function with respect to the current ensemble's predictions, iteratively reducing residual error in a stagewise manner (Friedman, 2001). This strategy allows GBDT models to capture complex non-linear relationships between software metrics and vulnerability labels without requiring manual feature engineering, and they remain largely unaffected by skewed input distributions and collinear features. Including XGBoost as the machine learning baseline is therefore methodologically necessary: it establishes a strong, well-validated lower bound that the deep learning architectures — TabM and FT-Transformer — must surpass to demonstrate that their additional architectural complexity is justified for this specific tabular task.

In this study, XGBoost receives as input the preprocessed feature vector of a JavaScript function — consisting of 35 static source code metrics when trained on the Ferenc et al. (2019) dataset, or 61 static and process metrics when trained on the Viszkok et al. (2021) dataset — both having been passed through a QuantileTransformer to normalize their distributions. At each boosting iteration m, XGBoost builds a new J-terminal node regression tree by fitting it to the pseudo-residuals — the negative gradient of the binary cross-entropy loss with respect to the current ensemble's log-odds predictions (Friedman, 2001). For a vulnerable function with label $y = 1$, the pseudo-residual is the difference between the true label and the model's current predicted probability, directing the next tree to focus its splits on functions the model is currently misfitting. Each tree partitions the feature space by greedily selecting the split variable and threshold that maximally reduces the weighted loss at each node. The resulting terminal node update values are then added to the running ensemble scaled by a learning

rate v, a shrinkage strategy shown by Friedman (2001) to substantially improve generalization over unshrunken updates. After M iterations, the final ensemble's log-odds score is passed through the sigmoid function and thresholded at 0.5 for binary classification.

To address the severe class imbalance in both datasets, XGBoost's *scale_pos_weight* parameter is set to the ratio of non-vulnerable to vulnerable training samples — approximately 7.10 for the Ferenc dataset and 7.37 for the Viszkok dataset — directly scaling the gradient contribution of each minority-class sample in line with the cost-sensitive objective described in Section 3.5. Systematic hyperparameter tuning is then applied via Optuna TPE over 100 trials on the fixed validation set, searching over nine key parameters including *n_estimators*, *max_depth*, *learning_rate*, *subsample*, and *colsample_bytree*, with PR-AUC as the optimization target and early stopping set to 50 rounds. Since the two datasets differ in feature dimensionality, each dataset is tuned independently under the same protocol, ensuring that the search explores split configurations appropriate to its respective feature space. The fully tuned model for each dataset is subsequently retrained on the corresponding training set and evaluated on the held-out test set.

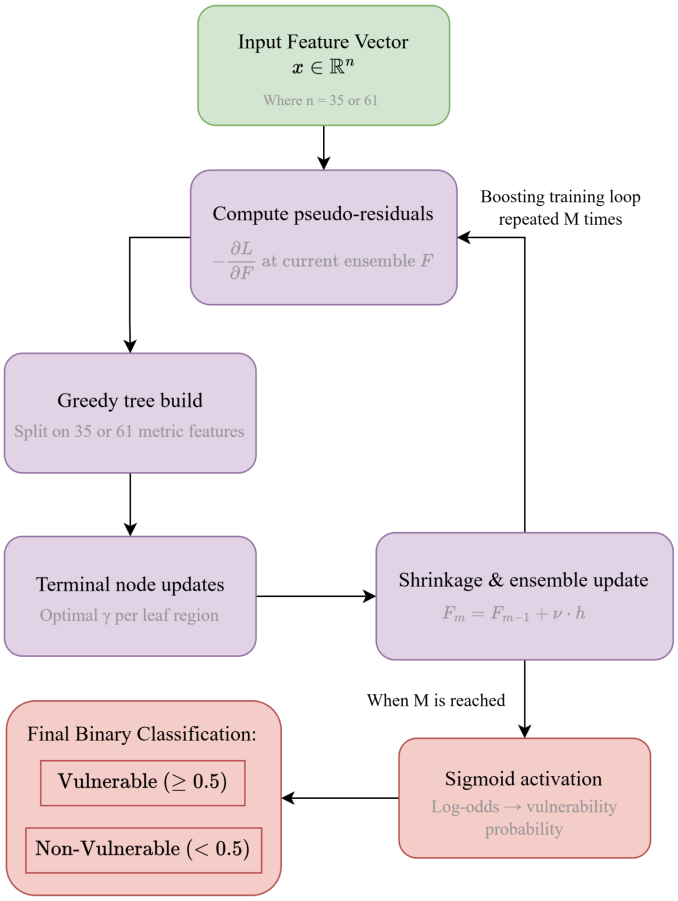


Figure 3.5 The End-to-End Pipeline of XGBoost

3.7 Hyperparameter Tuning

To ensure a fair and unbiased comparison, all three models are subjected to systematic hyperparameter tuning using an identical protocol. Tuning is performed exclusively on the training and validation sets, and the held-out test set is never accessed during this phase and would remain clean. Table 3.2 summarizes the tuning configuration for each model.

Table 3.3 Hyperparameter Tuning Configuration for All Three Architectures

Hyperparameter	GBDT (XGBoost)	FT-Transformer	TabM
Search Method	Optuna (TPE)	Optuna (TPE)	Optuna (TPE)
Num of Trials	100 trials	50 trials	50 trials
Validation Strategy	Fixed val set	Fixed val set	Fixed val set
Tuning Metric	PR-AUC	PR-AUC	PR-AUC
Early Stopping	50 Rounds	20 Epochs	20 Epochs
Key Parameters Tuned	n_estimators [50–2000] · max_depth [3–12] · learning_rate [0.001–0.3] · subsample [0.5–1.0] · colsample_bytree [0.5–1.0] · min_child_weight [1–10] · reg_alpha [0–5] · reg_lambda [0–5] · scale_pos_weight fixed at 7.37 / 7.10	num_layers [1–4] · d_token [64–512] · n_heads {4, 8} · attention_dropout [0.0–0.4] · ffn_dropout [0.0–0.4] · ffn_factor [1.0–4.0] · learning_rate [1e-5–1e-3] · weight_decay [1e-6–1e-3] · batch_size {256, 512, 1024}	num_layers [1–4] · hidden_dim [64–512] · dropout [0.0–0.5] · learning_rate [1e-5–1e-2] · weight_decay [1e-6–1e-3] · batch_size {256, 512, 1024} · k multiplier [4–64]

For the search method Optuna will be employed, specifically TPE or Tree-structured Parzen Estimator for all three models. It is a probabilistic model that searches and focuses on promising regions, making it more efficient than grid and random search. Every model will employ the same algorithms to ensure that no models are getting benefits from a smarter search energy.

The number of trials for machine learning and deep learning approaches are different, with 100 trials for XGBoost and 50 trials each for TabM and FT-Transformer. XGBoost trains around 5-15 seconds per trial, which means 100 trials costs around 8-25 minutes in total, depending on the testing environment. This number is necessary because XGboost has 9 interacting continuous

parameters where interactions like *learning_rate* × *n_estimators* and *subsample* × *colsample_bytree* require broader exploration. On the other hand, TabM and FT-Transformer cost 6–12 times more than XGBoost per trial. This means 50 trials is the practical ceiling that fits within the reasonable computing budget. In addition, both types of models have a similar number of settings to adjust (around 7 to 9 parameters). Because TPE learns from previous attempts to make smarter guesses, a 50 smart trials can cover as much useful ground as 100 trials on a faster model. Therefore, fairness does not mean giving every model the exact same number of tries, but it means giving each model an equally thorough search based on its computational cost.

For the validation strategy, a cross-validation approach within the tuning loop would be the ideal approach. However, given that the computational budget has already been allocated toward a sufficient number of trials — particularly for the deep learning models — applying cross-validation would multiply the cost beyond the reasonable limit. A fixed validation split is therefore adopted for all three models. Applying cross-validation exclusively to XGBoost while using a fixed split for TabM and FT-Transformer would grant an unfair advantage to the XGBoost model, receiving a more thorough validation. To maintain a fair and uniform tuning protocol, a single fixed validation set is used across all models.

As for the tuning metric, PR-AUC is selected for all models. Given the class imbalance of approximately 88:12 between non-vulnerable and vulnerable classes, ROC-AUC would be unsuitable, because it is heavily influenced by the number of true negatives. On the other hand, PR-AUC focuses on the precision and recall of the minority class, which is the class that actually matters in vulnerability detection. Applying the same metric across all three models ensures that all are being optimized toward the same objective, maintaining fairness in comparison.

Early stopping is also employed across all models to prevent overfitting and avoid unnecessary use of computational resources. For XGBoost, a patience of 50 rounds is applied, due to the large upper bound of the *n_estimators* search space, which requires sufficient rounds before reaching a meaningful plateau. As

for the deep learning models, TabM and FT-Transformer, early stopping is set at 20 epochs. This smaller patience value is appropriate given the cost per epoch is higher and the generally smoother convergence behavior observed in neural network training. In both cases, early stopping serves as a safeguard that allows the tuning process to stop a trial when there is no further improvement and conserve resources without compromising result quality.

3.8 Model Evaluation

Given the class imbalance in our dataset, specific evaluation metrics are needed to correctly measure the performance of our models. The focus should lean towards minimizing false negatives, as a false negative that passes into the deployment pipeline is considerably more dangerous than a false positive. Ensuring that the trade-off between model performance and computational resource consumption is reasonable is also an important consideration, as it allows us to identify which model delivers strong performance while remaining lightweight enough for application in a rapid CI/CD environment. To address both concerns, the evaluation metrics are split into two distinct categories: performance metrics and efficiency metrics.

3.8.1 Performance Metrics

PR-AUC measures the area under the Precision-Recall curve across all classification thresholds for the minority class. It summarizes the full precision-recall tradeoff on the vulnerable class without being inflated by the majority class. On the other hand, ROC AUC is susceptible to inflation caused by high true negative counts, which can make a poorly performing model appear deceptively strong. Therefore, PR-AUC is chosen over ROC-AUC as the primary performance metric.

$$PR\text{-}AUC = \sum_{i=1}^{n} (R_i - R_{i-1}) \cdot P_i \qquad (3.3)$$

where P_i is the precision at threshold i, derived from:

$$Precision = \frac{TP}{TP + FP} \quad (3.4)$$

and R_i is the recall at threshold i, derived from:

$$Recall = \frac{TP}{TP + FN} \quad (3.5)$$

F2-Score measures the *F-beta* at $\beta=2$, weighting recall twice as heavily as precision. This evaluation metric is important for vulnerability detection because a false negative or a missed vulnerability is much more dangerous than a false positive or a non-vulnerable that were flagged as vulnerable.

$$F2\text{-}Score = \frac{5 \times Precision \times Recall}{(4 \times Precision) + Recall} \quad (3.6)$$

MCC or Matthews Correlation Coefficient is a correlation coefficient, which uses all four cells of the confusion matrix. This evaluation metric is necessary for imbalanced binary classification. It complements PR-AUC by capturing TP, TN, FP, FN simultaneously, avoiding the PR-AUC blind spot of ignoring true negatives.

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (3.7)$$

3.8.2 Efficiency Metrics

Inference latency captures the mean ms per sample on the test set using CPU, single-thread. This metric is used as the direct measure of the CI/CD suitability. It is measured on the CPU to reflect realistic pipeline deployment without GPU dependency.

Retraining time captures the wall-clock in minutes to retrain the model from scratch on the training set. In rapid CI/CD, models are retrained on every code push or periodically. A model that takes a long time to retrain creates pipeline bottlenecks regardless of inference speed.

Model size of the trained model will also be measured in MegaBytes (MB). It directly impacts storage in CI/CD registries. This is especially relevant for FT-Transformer which can easily exceed 100 MB.

Peak Memory (RAM) captures the peak RAM usage during inference calculated in MegaBytes (MB). It measures whether a model can run inside a standard CI runner which is typically 2-8 GB. FT-Transformer's 500–2000 MB footprint may exceed lightweight runner limits.

3.9 Hardware and Software Environment

All experiments are conducted in a consistent computational environment to ensure reproducibility. The primary software stack consists of Python 3.12.4 as the programming language, PyTorch 2.5.1 as the deep learning framework for the TabM and FT-Transformer implementations, the XGBoost library, Scikit-learn for preprocessing utilities, Optuna for hyperparameter optimization, and Pandas and NumPy for data manipulation.

The hardware environment used for training consists of a workstation equipped with an NVIDIA GeForce RTX 4060 GPU with 8GB VRAM, 16 GB of system RAM, and an Intel Core i7-14650HX processor. All experiments are run with a fixed global random seed (seed = 42) applied to all relevant libraries (NumPy, PyTorch, Python random module) to guarantee reproducibility of results.

Chapter 4
RESEARCH SCHEDULE

This research is designed to be executed over a period of approximately 7 months, commencing immediately after the proposal defense in May 2026 and concluding with the final manuscript submission in December 2026. The technical implementation is divided into four primary phases, followed by manuscript drafting and examination milestones.

The schedule accounts for local GPU training time constraints and allocates specific buffer weeks for iterative reviews by the academic advisor. The specific timeline and phase breakdowns are detailed in Table 4.1 below.

Table 4.1 Proposed Research Schedule

Milestone / Activity	**May '26**	**Jun '26**	**Jul '26**	**Aug '26**	**Sep '26**	**Oct '26**	**Nov '26**	**Dec '26**
1. Proposal & Revisions								
Proposal Submission & Defense	X							
Post-Defense Revisions	X	X						
2. Technical Implementation								
Phase 1: Data Prep & Class Weights		X						
Phase 2: Baseline Models (XGBoost/ FT-Transformer)		X						
Phase 3: TabM Implementation & Tuning			X					

Milestone / Activity	May '26	Jun '26	Jul '26	Aug '26	Sep '26	Oct '26	Nov '26	Dec '26
Phase 4: Evaluation & Time Benchmarks			X	X				
3. Manuscript & Examination								
Chapter 4 (Results) Drafting & Review				X				
Chapter 5 (Conclusion) & Finalization					X			
Seminar Hasil (Results Seminar)					X	X		
Final Thesis Defense (Pendadaran)							X	
Final Revisions & Yudisium Submission								X

REFERENCES

Anwar, A., Khormali, A., Choi, J., Alasmary, H., Choi, S., Salem, S., Nyang, D., & Mohaisen, D, 2020, Measuring the Cost of Software Vulnerabilities, ICST Transactions on Security and Safety, 7(23), 164551, https://doi.org/10.4108/eai.13-7-2018.164551.

Cheirdari, F., & Karabatis, G, 2018, Analyzing False Positive Source Code Vulnerabilities Using Static Analysis Tools, 2018 IEEE International Conference on Big Data (Big Data), 4782–4788. https://doi.org/10.1109/BigData.2018.8622456.

Elkan, C., 2001, The Foundations of Cost-Sensitive Learning, Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI '01), 973–978, https://dl.acm.org/doi/10.5555/1642194.1642224.

Ferenc, R., Hegedus, P., Gyimesi, P., Antal, G., Ban, D., & Gyimothy, T., 2019, Challenging Machine Learning Algorithms in Predicting Vulnerable JavaScript Functions, 2019 IEEE/ACM 7th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering (RAISE), 8–14, https://doi.org/10.1109/RAISE.2019.00010.

Friedman, Jerome., 2000. Greedy Function Approximation: A Gradient Boosting Machine, The Annals of Statistics. 29. 10.1214/aos/1013203451.

Gorishniy, Y., Kotelnikov, A., & Babenko, A., 2025, TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling (arXiv:2410.24210), arXiv, https://doi.org/10.48550/arXiv.2410.24210.

Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A., 2021, Revisiting Deep Learning Models for Tabular Data (arXiv:2106.11959), arXiv, https://doi.org/10.48550/arXiv.2106.11959.

Kalouptsoglou, I., Siavvas, M., Kehagias, D., Chatzigeorgiou, A., & Ampatzoglou, A., 2022, Examining the Capacity of Text Mining and Software Metrics in Vulnerability Prediction, Entropy, 24(5), 651. https://doi.org/10.3390/e24050651.

Medeiros, N., Ivaki, N., Costa, P., & Vieira, M., 2020, Vulnerable Code Detection Using Software Metrics and Machine Learning, IEEE Access, 8, 219174–219198, https://doi.org/10.1109/ACCESS.2020.3041181.

Mosolygo, B., Vandor, N., Antal, G., Hegedus, P., & Ferenc, R., 2021, Towards a Prototype Based Explainable JavaScript Vulnerability Prediction Model, 2021 International Conference on Code Quality (ICCQ), 15–25, https://doi.org/10.1109/ICCQ51190.2021.9392984.

Pakshad, P., Shameli-Sendi, A., & Khalaji Emamzadeh Abbasi, B., 2023, A security vulnerability predictor based on source code metrics, Journal of Computer Virology and Hacking Techniques, 19(4), 615–633, https://doi.org/10.1007/s11416-023-00469-y.

Pan, Z., Shen, W., Wang, X., Yang, Y., Chang, R., Liu, Y., Liu, C., Liu, Y., & Ren, K., 2024, Ambush From All Sides: Understanding Security Threats in Open-Source Software CI/CD Pipelines, IEEE Transactions on Dependable and Secure Computing, 21(1), 403–418,

https://doi.org/10.1109/TDSC.2023.3253572.
Sakho, A., Malherbe, E., & Scornet, E., 2024, Do we need rebalancing strategies? A theoretical and empirical study around SMOTE and its variants (arXiv:2402.03819), arXiv, https://doi.org/10.48550/arXiv.2402.03819.
Sheneamer, A., 2024, Vulnerable JavaScript functions detection using stacking of convolutional neural networks, PeerJ Computer Science, 10, e1838, https://doi.org/10.7717/peerj-cs.1838.
Shwartz-Ziv, R., & Armon, A., 2022, Tabular data: Deep learning is not all you need, Information Fusion, 81, 84–90, https://doi.org/10.1016/j.inffus.2021.11.011.
Viszkok, T., Hegedűs, P., & Ferenc, R., 2021, Improving Vulnerability Prediction of JavaScript Functions Using Process Metrics (arXiv:2105.07527), arXiv, https://doi.org/10.48550/arXiv.2105.07527.
Ye, H.-J., Liu, S.-Y., Cai, H.-R., Zhou, Q.-L., & Zhan, D.-C., 2025, A Closer Look at Deep Learning Methods on Tabular Datasets (arXiv:2407.00956), arXiv, https://doi.org/10.48550/arXiv.2407.00956.
Zagane, M., Abdi, M. K., & Alenezi, M., 2020, Deep Learning for Software Vulnerabilities Detection Using Code Metrics. IEEE Access, 8, 74562–74570, https://doi.org/10.1109/ACCESS.2020.2988557.

ATTACHMENTS

Ferenc Dataset Summary Statistic Table

(n = 12,125 | 35 features | Vuln: 1,496 / 10,629)

Metric	Mean	Std Dev	Min	Median	Max
Complexity					
McCC	3.071	4.789	1.000	2.000	137.0
NL	0.765	1.175	0.000	0.000	14.000
NLE	0.705	1.019	0.000	0.000	9.000
NUMPAR	1.161	1.260	0.000	1.000	13.000
CYCL	2.674	3.581	1.000	1.000	121.0
PARAMS	1.191	1.263	0.000	1.000	13.000
CYCL_DENS	67.212	51.633	0.000	50.000	900.0
Size / LOC					
LLOC	8.244	17.737	1.000	4.000	1071.0
LOC	10.669	28.641	1.000	4.000	1244.0
NOS	5.168	9.934	0.000	2.000	487.0
TLLOC	16.381	95.350	1.000	4.000	7225.0
TLOC	20.927	124.8	1.000	4.000	8199.0
TNOS	12.044	68.287	0.000	3.000	4923.0
Halstead					
HOR_D	6.307	4.706	0.000	5.000	37.000
HOR_T	49.515	200.2	0.000	10.000	1674.0
HON_D	25.313	82.235	0.000	9.000	989.0
HON_T	57.229	214.5	0.000	13.000	2415.0
HLEN	106.7	414.2	0.000	23.000	3586.0
HVOC	31.620	84.041	0.000	14.000	1010.0
HDIFF	6.458	7.238	0.000	3.900	98.118
HVOL	787.0	3857.8	0.000	85.837	35,789
HEFF	13,954	74,490	0.000	342.1	917,608
HBUGS	0.262	1.286	0.000	0.029	11.930
HTIME	775.2	4138.3	0.000	19.005	50,978

Comment					
CD	0.042	0.117	0.000	0.000	0.966
CLOC	1.365	7.891	0.000	0.000	259.0
DLOC	0.363	3.040	0.000	0.000	134.0
TCD	0.040	0.106	0.000	0.000	0.966
TCLOC	2.526	20.333	0.000	0.000	736.0
Clone					
CC	0.129	0.331	0.000	0.000	1.000
CCL	0.225	0.887	0.000	0.000	22.000
CCO	11.465	49.696	0.000	0.000	358.0
CI	0.251	1.159	0.000	0.000	48.000
CLC	0.162	0.364	0.000	0.000	1.000
CLLC	0.000	0.000	0.000	0.000	0.000

Note: Std Dev = standard deviation. All values rounded to 3 decimal places where applicable.

Viszkok Dataset Summary Statistic Table

(n = 8,038 | 58 features | Vuln: 960 / 7,078)

Metric	Mean	Std Dev	Min	Median	Max
Complexity					
CYCL	2.572	3.412	1.000	1.000	121.0
CYCL_DENS	67.985	50.057	0.000	53.846	900.0
McCC	2.876	4.662	1.000	1.000	137.0
NL	0.680	1.099	0.000	0.000	14.000
NLE	0.631	0.977	0.000	0.000	9.000
NUMPAR	0.986	1.098	0.000	1.000	13.000
PARAMS	0.967	1.077	0.000	1.000	13.000
Size / LOC					
LDC	9.574	107.7	0.000	0.000	8176.0
LLDC	7.630	91.162	0.000	0.000	7207.0
LLOC	7.223	18.647	1.000	3.000	1071.0
LOC	9.315	29.467	1.000	3.000	1244.0
NOS	4.691	10.003	0.000	2.000	487.0
TLLOC	14.026	100.4	1.000	3.000	7225.0
TLOC	17.725	124.9	1.000	3.000	8199.0
TNOS	11.169	74.399	0.000	2.000	4923.0
Halstead					
HBUGS	0.355	1.566	0.000	0.028	11.930
HDIFF	6.340	7.058	0.000	3.750	98.118
HEFF	18,650	89,938	0.000	311.3	917,608
HLEN	136.9	502.8	0.000	22.000	3586.0
HON_D	31.919	99.636	0.000	9.000	989.0
HON_T	72.900	260.1	0.000	13.000	2415.0
HOR_D	6.297	4.692	0.000	5.000	31.000
HOR_T	64.009	243.2	0.000	10.000	1674.0
HTIME	1036.1	4996.5	0.000	17.293	50,978
HVOC	38.216	101.6	0.000	13.000	1010.0

HVOL	1063.5	4697.4	0.000	84.000	35,789
Comment					
CD	0.033	0.100	0.000	0.000	0.917
CLOC	1.020	6.717	0.000	0.000	204.0
DLOC	0.191	1.235	0.000	0.000	32.000
TCD	0.033	0.093	0.000	0.000	0.917
TCLOC	1.813	16.430	0.000	0.000	736.0
Clone					
CC	0.348	0.474	0.000	0.000	1.000
CCL	0.715	3.029	0.000	0.000	224.0
CCO	156.2	390.8	0.000	0.000	3642.0
CI	0.747	3.432	0.000	0.000	254.0
CLC	0.375	0.481	0.000	0.000	1.000
CLLC	0.375	0.481	0.000	0.000	1.000
Process					
AVGNOAL	15.050	111.7	1.000	3.000	8199.0
AVGNODL	0.560	17.458	0.000	0.000	1545.0
AVGNOEMT	3398.2	2261.9	0.556	3094.0	11,606
AVGNOML	0.258	1.826	0.000	0.000	114.2
AVGTBC	10.579	59.520	0.000	0.000	935.0
MNOAL	17.465	120.5	1.000	3.000	8199.0
MNODL	0.717	17.823	0.000	0.000	1550.0
MNOEMT	3486.0	2243.6	1.000	3094.0	11,606
MNOML	0.453	4.151	0.000	0.000	287.0
Change History					
CChurn	18.112	125.2	-16.000	3.000	8199.0
NII	0.367	1.631	0.000	0.000	40.000
NOADD	1.193	1.588	1.000	1.000	97.000
NOCC	0.109	0.665	0.000	0.000	21.000
NOCHG	1.360	2.520	1.000	1.000	137.0
NOContr	1.068	0.373	1.000	1.000	9.000

NODEL	0.108	0.801	0.000	0.000	46.000
NOI	0.373	1.019	0.000	0.000	18.000
NOMOD	0.249	1.946	0.000	0.000	102.0
SOADD	19.330	138.0	1.000	3.000	8199.0
SODEL	1.217	35.587	0.000	0.000	3090.0
SOMOD	0.963	11.950	0.000	0.000	675.0

Note: Std Dev = standard deviation. All values rounded to 3 decimal places where applicable.